End of Year Competition
CanSat 2026

Team Name: Arceus

System Operations

Prepared by: Adam

Concept of Operations

1. Prelaunch	- Cansat is powered on - Sensors, telemetry, avionics, and battery are checked - Secure payload onto drone U-mount interface - Check parachute and release mechanism are armed and present
2. Ascent	- Drone carries CANSAT and begins liftoff - During this phase, attachment must remain safe, stable, secure - Ascend to 120m while maintaining nominal attachment
3. Release	- At 120m, CANSAT separates from the drone through the release mechanisms after 10s of hovering - CANSAT should fully detach, with all drone and CANSAT parts intact
4. Initial descent detection	- System detects descent via barometric pressure sensor readings - Descent categorized by <=1m decrease in height within 3 consecutive readings - CANSAT parachute release system instantaneously deploys, ensuring speed doesn't exceed 3m/s +/- 1m/s threshold at any moment

Concept of Operations

5. Telemetry phase	- Once descent starts, CANSAT should transmit data packets with altitude, GPS position, rotational velocity, and packet sequence number - Packet to be delivered at minimum three times
6. Controlled descent	- CANSAT descends at 3m/s +/- 1m/s - Remains stable under pitch/roll (<=45° from nadir) and yaw (<=1 rev/s) - Parachute remains attached and does not tear, tangle, or change from the intended vertical orientation relative to the ground during descent
7. Touchdown	- CANSAT lands successfully in a vertically oriented and upright position - CANSAT remains intact and lands with high stability within a reasonable area around the drone-drop site
8. After touchdown	- CANSAT is recovered, switched off, and rearmed - Telemetry packets are used for verifying mission success

Weight Requirements

Component	Qty	Mass (g)
Primary flight CPU	1	7
Pressure sensor	1	1.18
GPS	1	17
IMU	1	1.8
Telemetry radio module	1	4

Component	Qty	Mass (g)
Antenna	1	2
Battery	2	111
V regulator and P switch	1	8
PCB	1	12
Release servo	1	9
Wiring	N/A	20

Weight Requirements Cont'd

Component cont'd	Qty cont'd	Mass (g) cont'd
Fasteners	N/A	10
Landing protection	N/A	8
Parachute	1	56
Housing	1	90
Release mech hardware	N/A	30
Buffer room for error	N/A	20
TOTAL		***406.98***

Component weight was derived from reasonable estimates or from the mass of each specific part (reference the part lists shown later in the PDR)

Mechanical Subsystem Design

Prepared by: Adam + *******

Dimensional Requirements

- The Satellites Bodies dimensions will be 131.25 mm (W) x 210 mm (H)
- The Release Mechanisms dimensions will be 143.75 (mm) x 230 (mm)

Parachute Design

DESIGN CHOICE 1

- **Flat circular chute + spill hole**

- Easy to manufacturer given the basic shape and connection methods via fabric + nylon
- Relatively easy to calculate dimensions
- Spill hole provides much needed stability

- Harder to pack into small hull
- Complex deployment
- Heavier than alternatives

Parachute Design Cont'd

DESIGN CHOICE 2

- **Streamers**

- Much easier to manufacture and very simple to incorporate
- Very light and compact for easy packing
- Deployment is rather simpler compared to traditional chutes

- Very low stability (sensitive to flutters + oscillations)
- Usually leads to faster landing
- Much harder to land upright

Parachute Design Cont'd

DESIGN CHOICE 1 IS CHOSEN

Flat circular chute + spill hole

- Despite the larger, heavier, and slightly complex nature, these type of parachutes are much more predictable aerodynamically, which includes stable, slow, and upright landing that better suit mission parameters
- Compare this to the rather simple, but poor mission-fulfillment behavior streamers experience with unstable and fast landing attributes, which is not an acceptable tradeoff

Descent Speed Calculations

Targets: **V = 3 m/s**, m = 406.98g, spill hole diameter of 10%, 6 shroud lines that are 120% longer than the given parachute diameter, C_d = 0.8 (average nylon parachute C_d value)

Assumptions: No wind, assume atm conditions, and assume weather conditions are negligible

Justifications of above: Simplify design and speed calculations while still keeping reasonable precautions for manufacturability and performability

$mg = (½)\rho C_d A V^2$

$A = 2mg/(\rho C_d V^2)$

A = 0.91

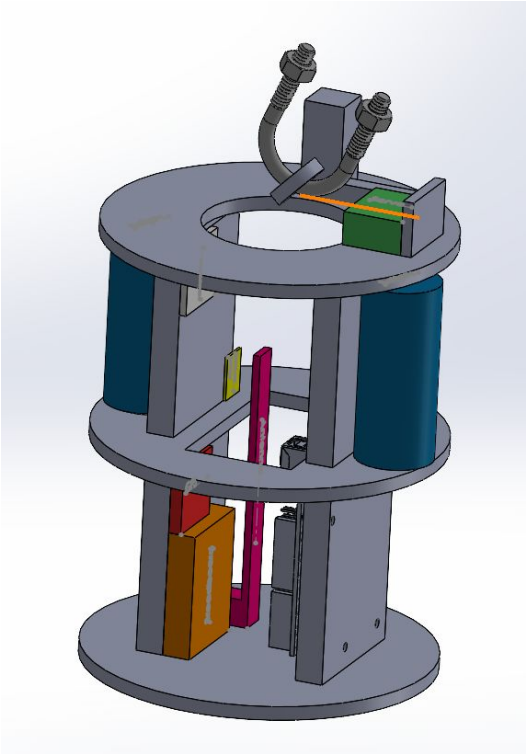
$D = \sqrt{(4A/\pi)} = 1.07m$

Spill hole = 10% of diameter, thus; **$D_{final} = 1.07m + 0.107m = 1.18m$**

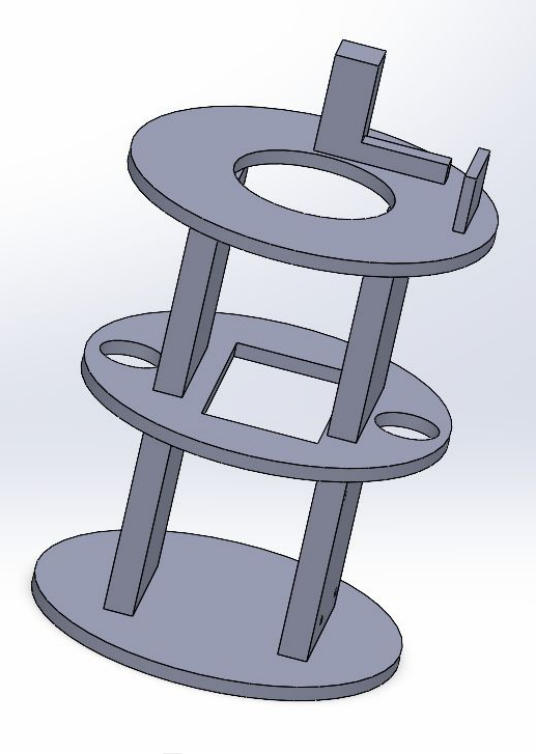
Assume: **$L_{shroud} = 1.2(D_{final}) = 1.42m$**

L_{total} = 1.42m * 6m = 8.52m + 1 m to account for knots and trims = 9.52m

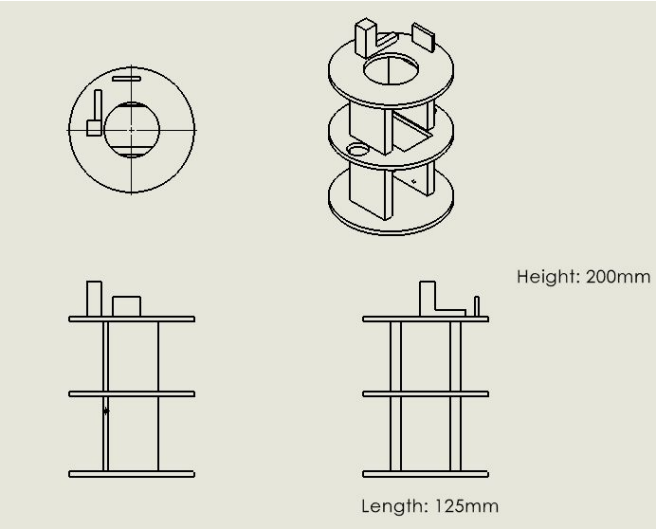
CAD Images (NOT FINAL)



Assembly

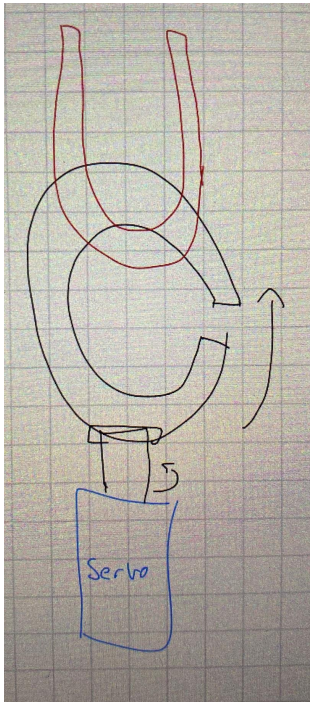


Base



Drawing

Release Mechanism



Release mechanism is a ring with a cutout in the side, which will be rotated by the servo.

Not shown fully in CAD*

(still working out the proper format of how everything will be connected)

Stable Landing Methods

Methodology	Justification
Low centre of mass	Position battery and other heavy components on the lowest compartment and at around the centreline, which helps keep a low COM and high stability during descent (maintains pitch/roll control)
Symmetric design	Keep the design symmetric in terms of the hull, while also ensuring the design is symmetric in terms of mass to prevent unneeded instability (no platform or side of the CANSAT should significantly outweigh the other side to maintain controlled descent)
Centred parachute + spill hole	Parachute attachment should be along the centreline as well to reduce tilt and unbalanced loads during descent, particularly with a spill hole, to ensure no significant instability stems from parachute position
Rigid, cushioned base via foam	A base-cushioned foam helps reduce shocks and stop rebound during impact, minimizing the chance for side-landing
Slower descent via parachute	Slow descent helps reduce general instability from aerodynamic forces via air resistance

Internal Landing Protection

1

BATTERY MOUNTING

- Mount battery at the lower portion/level of the CANSAT
- Provides low but centred COM, reduce component shifting
- Thus stops tipping, bouncing, and side landing for reduce unintended impacts on the side

2

COMPONENT MOUNTING

- Use standoffs and screws for mounting PCBs, servos, and nearly all components
- Create slots for parts to friction-fit in
- Stops component shifting an unintended internal damage

3

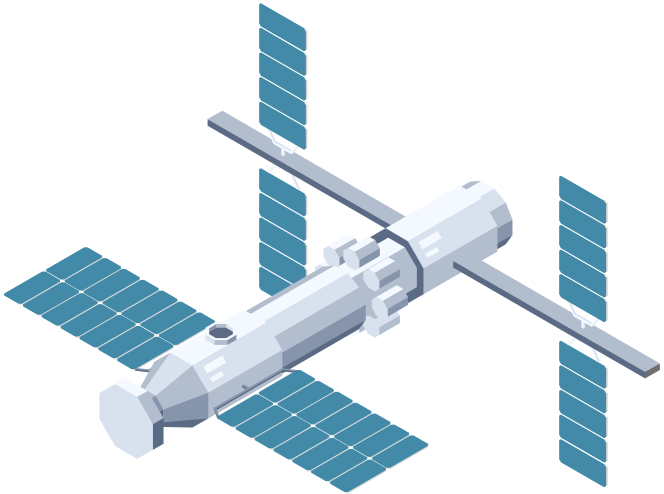
PARACHUTE USAGE

- Reduce impact speed, lowering the raw impact force as a whole
- Keep descent at 3m/s consistently

External Landing Protection

Reinforced bottom plate (115% - 130% thicker than surrounding walls) accompanied with fillets for round edge connections to the rest of the hull

★ Better load distribution and reduce chance of cracks along high-stress CANSAT hull



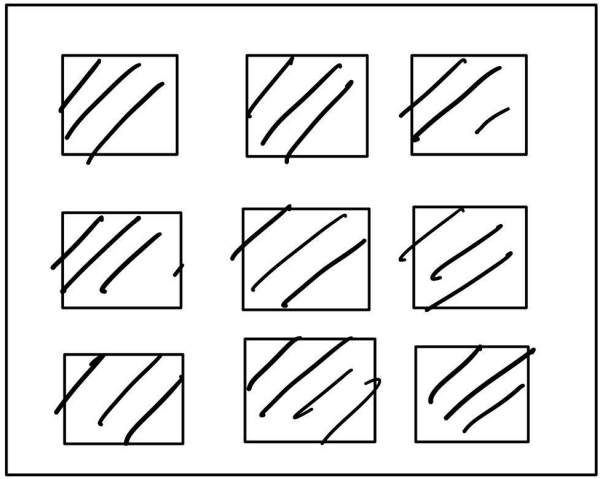
External Landing Protection Cont'd

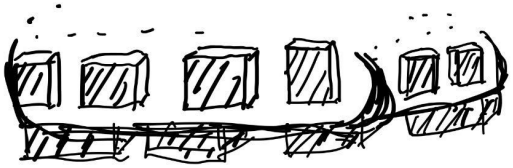
Thick foam padding on the base along the bottom/lower perimeter impact-prone regions of the payload to:

- ★ Absorb impact significantly, and slows impact over a larger Δt
- ★ Reduce shock transmission to components higher up
- ★ Keep a structurally sound base
- ★ Good in case of accidental side impact
- ★ Low in terms of weight, so low weight-tradeoff

NOTE: square foam pads will be placed and spaced apart like tiles around the lower edges/sides for easier manufacturing, placement, integration with the CANSAT hull, and adhesion via adhesives at the bottom

Rough Sketch of Foam Pad Placement





Electrical Subsystem Design

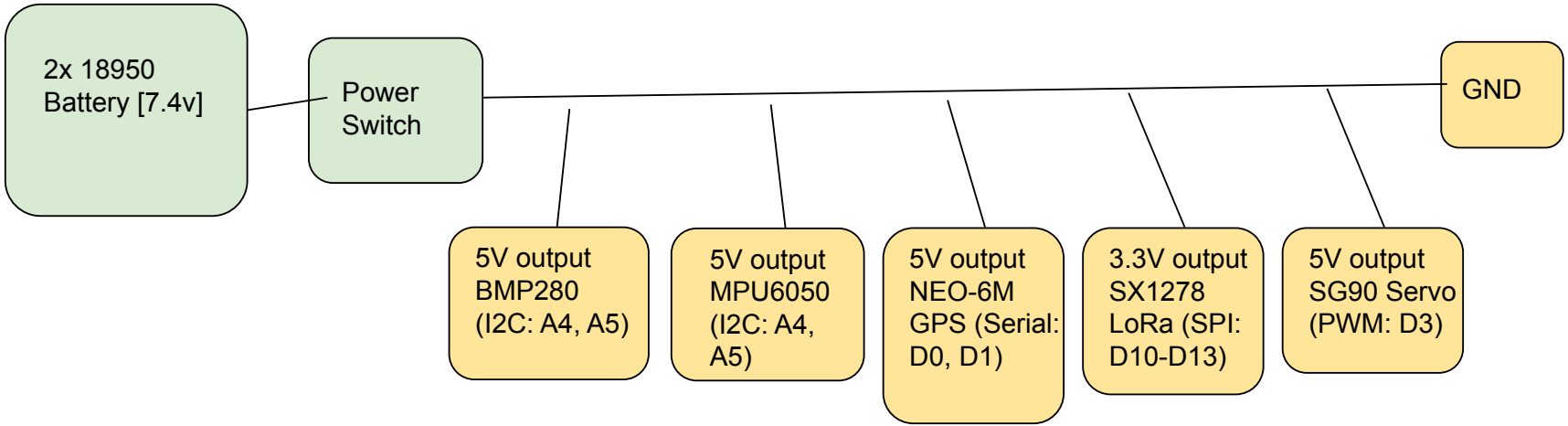
Prepared by: ******* + Adam**

Overview

Microcontroller (Arduino)	Main controller for all sensores, collecting and processing data while a maintaining communication
Sensors (GPS, Pressure, IMU)	Measure altitude, provide location, and measure acceleration in terms of pitch, roll, and yaw
Radio Module for telemetry + Antenna	Sends telemetry data to the ground station. This includes altitude, GPS, rotation, and packet numbers. The antenna is integrated with NRF24L01 to allow for wireless transmissions
Battery + Power control system	Supplies Power to all electrical devices/components. The Power switch allows for easy access with an on/off switch
Mechanisms	Consists of a SG90 Micro Servo, which can be used as a detaching mechanism from the drone and as a deployment system for the parachute
Breadboard, Wiring/Connectors/Resistors	Additional parts included to help with circuits, connections between components and providing stability

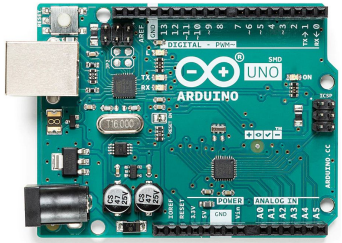
Schematic Design

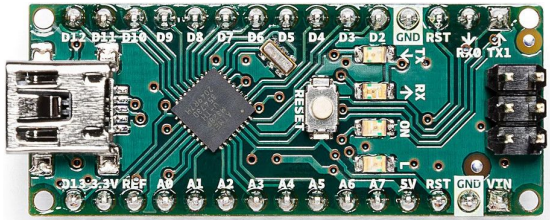
- Rudimentary schematic to build from after PDR
- To be continued by Adam

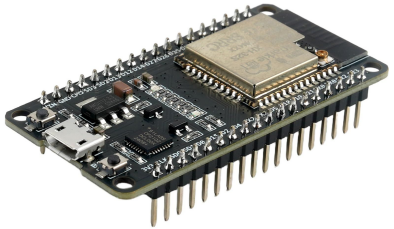


Avionics Processor Specification

Processor	Clock speed	Operating voltage	GPIO Pins	Memory	Cost	Dimensions	Mass
Arduino Uno	16 MHz	5V	14D, 6A	32 KB	$28	69mmx53mm	25g
Arduino Nano	16 MHz	5V	14D, 8A	32 KB	$25	45mmx18mm	7g
ESP32	160-240 MHz	3.3V	30	448 KB	$10	51mmx21mm	10g







Data Interfaces

Types of pins:

Digital Pins (GPIO)
Quantity: 14 (D0-D13)
Function: Can be input or output, used to read sensors or control servo
Signal: 2 states-0 (Low, 0V) or 1 (High, 5V)

Analog Input Pins
Quantity: 6 (A0-A5)
Function: Input pins, reads continuous signals from pressure and temperature sensors
Signal: gradual- 0-1023 (0-5V)

Pulse Width Modulation (PWM) Pins:

Pins: D3, D5, D6, D9, D10, D11
Type: Digital output with PWM capabilities
Function: Provide analog like output used for servo and actuator control

Digital: input or output
Analog: Input
PWM: output

Sensors

Barometric Pressure Sensor	The sensor used is a BMP280 and its job is to measure altitude and detect descent using pressure changes.
GPS Module	The GPS Module used is a NEO-6M, this provides the position of the satellite, both longitude and latitude.
Inertial Measurement Unit (IMU)	The IMU used is an MPU-6050, the goal of this sensor is to measure the acceleration of the sattletie and its rotational rate. Those being its pitch, roll, and yaw.

Payload Antenna

NRF24L01 Antenna

Range: 300 - 1000 m

Radiation Pattern: Donut (Torus) shape

Frequency: 2.4 GHz

Voltage requirement: 3.3V

Peak current: 100mA

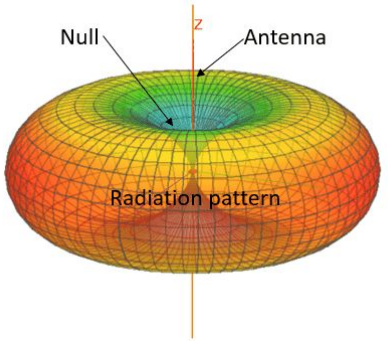
Communication

Serial Peripheral Interface (SPI) protocol

- Wired connection to communicate with arduino

Half-duplex transmission

- Receives and transmits data



Battery Design

Model	Voltage Range	Capacity	Current output	Cost	Weight	Dimensions	Power
2x 18650 Li-ion	6 - 8.4V	3000mAh	High	$15	110 g	65mmx36mm	11.1 Wh
9V	7 - 9V	500mAh	low	$12	45 g	48mmx26mmx17 mm	4.5Wh
4x AA (NiMH)	4 - 5.6V	2000mAh	Medium	$23	120g	50mmx60mmx15 mm	9.6 Wh

2, 18650 Li-ion batteries were chosen due to its high voltage range, capacity, and cost effectiveness. The battery is Li-ion which complies with requirement E6, and will supply enough power for all electrical components. Batteries will be connected to breadboard and secured into the cansat thought the avionics bay.

Software Subsystem Design

Prepared by: *******, *********, Adam, and *********

Overview: Summary

Main Software Tasks:

Sensor Data Acquisition

- Sample BMP280 (pressure/altitude) at 10 Hz
- Read NEO-6M GPS position at 1 Hz
- Monitor MPU6050 (gyroscope/accelerometer) at 10 Hz
- Store data in buffers for processing

State Management

- Implement 6-state finite state machine
- Handle transitions based on altitude and sensor readings
- Manage timers for release mechanism (10s delay at 120m)

Descent Detection

- Monitor 3 consecutive altitude readings
- Detect altitude decrease ≥1 meter
- Trigger telemetry transmission when descent confirmed

Telemetry Transmission

- Format packet with: Altitude, GPS, Rotation rate, Sequence number
- Transmit via SX1278 LoRa module
- Send 3 times for reliability (requirement S3)

Overview: Summary

Main Software Tasks:

Release Mechanism Control

- Activate SG90 servo at 10 seconds after reaching 120m
- PWM control to open release latch
- Return servo to neutral position

Ground Communication

- Maintain LoRa link with ground station
- Handle packet acknowledgments
- Log all data for post-flight analysis

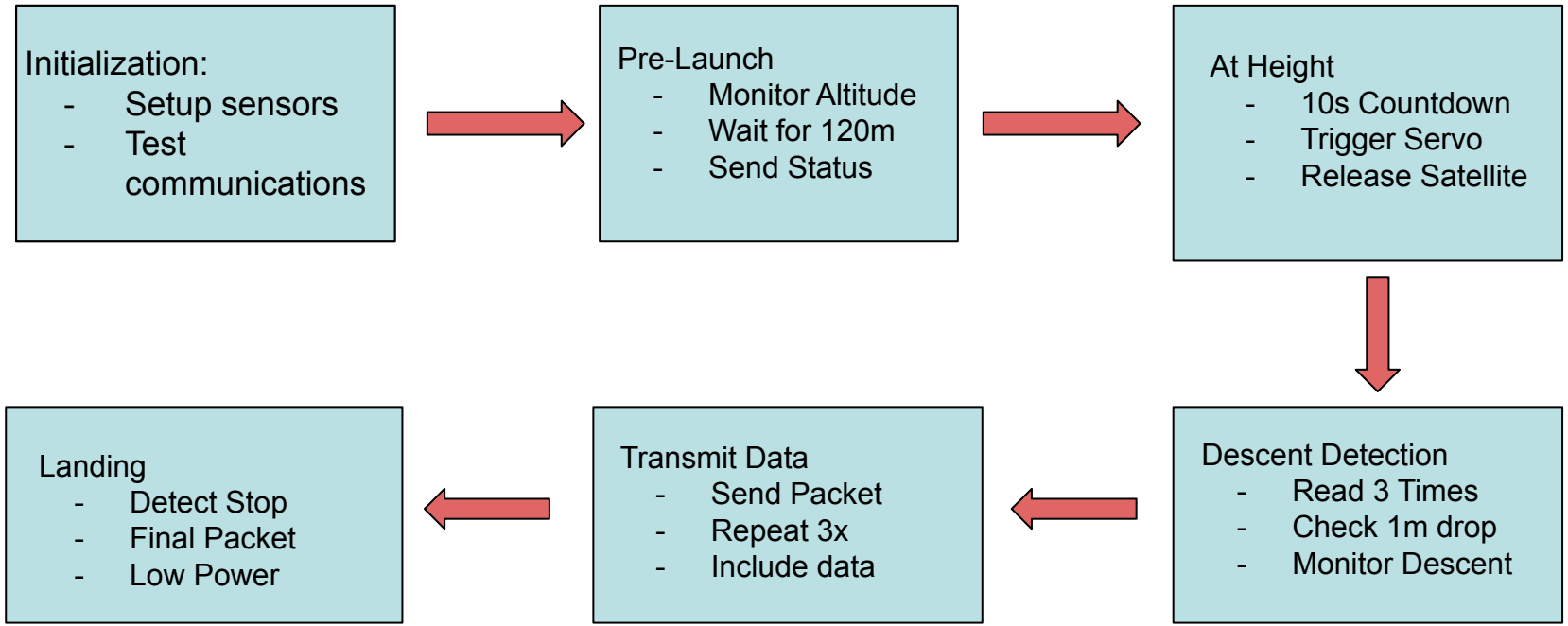
Programming Language: Arduino C/C++

Development Environment: Arduino IDE

Key Libraries:

- Wire.h (I2C for BMP280, MPU6050)
- SoftwareSerial or HardwareSerial (GPS communication)
- LoRa.h or RadioHead.h (SX1278 LoRa control)
- Adafruit_BMP280.h (pressure sensor)
- TinyGPS++.h (GPS parsing)
- MPU6050.h (IMU control)
- Servo.h (release mechanism)

Overview: Payload Flowchart



Payload State Diagram

State 1: Initialization

Sensors: Test all sensors
Sampling: One-time startup
Comms: Establish ground link
Data Log: System startup log
Mechanisms: Servo to neutral (0°)
Power: All systems active
Decision: All sensors OK?

State 2: Pre-launch

Sensors: BMP280, GPS, MPU6050
Sampling: 10 Hz (all sensors)
Comms: Status packets at 1 Hz
Data Log: Continuous logging
Mechanisms: Servo idle
Power: Normal operation (~212 mA)
Decision: Altitude >= 120m?

State 3: At Height

Sensors: Continue all sensors
Sampling: 10 Hz (monitoring)
Comms: Countdown status sent
Data Log: Record exact release time
Mechanisms: 10s timer → Servo to 90°
Power: Servo spike to ~350 mA
Decision: 10 seconds elapsed?

State 4: Descent Detect

Sensors: BMP280 (critical), GPS, MPU
Sampling: 10 Hz (100ms intervals)
Comms: Listening mode (LoRa RX)
Data Log: Buffer last 3 altitude reads
Mechanisms: Servo returns to 0°
Power: Normal (~212 mA)
Decision: Alt[0] - Alt[2] <= -1.0m?

State 5: Transmit

Sensors: Continue sampling
Sampling: 10 Hz (still collecting)
Comms: Transmit packet 3 times (LoRa TX)
Data Log: Log transmission timestamps
Mechanisms: None
Power: TX spike to ~300 mA (brief)
Decision: 3 packets sent?

State 6: Landing

Sensors: BMP280, MPU6050 only
Sampling: 1 Hz (reduced rate)
Comms: Final status packet, then idle
Data Log: Landing timestamp, impact
Mechanisms: None
Power: Reduced (~150 mA, GPS off)
Decision: Altitude stable for 5s?

Payload State Diagram

Sampling Rates Summary

Sensor	Pre-Launch	At Height	Descent	Transmit	Landing
BMP280	10 Hz	10 Hz	10 Hz	10 Hz	1 Hz
GPS	1 Hz	1 Hz	1 Hz	1 Hz	OFF
MPU6050	10 Hz	10 Hz	10 Hz	10 Hz	1 Hz
LoRa TX	1 Hz (status)	1 Hz	OFF	3 bursts	1 packet

Height State - Logic Flow

Altitude Measurement:

The BMP280 sensor measures atmospheric pressure and converts it to altitude using the barometric formula:

```
Altitude (m) = 44330 × [1 - (P / P₀)^0.1903]

Where:
- P = Current pressure (hPa)
- P₀ = Sea level reference pressure (1013.25 hPa)
```

Ground Level Calibration:

At startup, the system records the ground-level pressure as a reference:
- `ground_pressure = BMP280.readPressure();`
- All altitude calculations are relative to this baseline
- `relative_altitude = calculated_altitude - ground_altitude;`

Height State - Pseudocode

```cpp
// GLOBAL VARIABLES
enum State {INIT, PRE_LAUNCH, AT_HEIGHT, DESCENT_DETECT, TRANSMIT, LANDIN
State current_state = INIT;

float ground_altitude = 0.0;
float current_altitude = 0.0;
const float TARGET_ALTITUDE = 120.0;  // meters
const unsigned long RELEASE_DELAY = 10000;  // 10 seconds in milliseconds
unsigned long height_reached_time = 0;
bool release_triggered = false;

Servo releaseServo;
const int SERVO_PIN = 3;
const int SERVO_OPEN = 90;    // Release position
const int SERVO_CLOSED = 0;   // Holding position

// MAIN LOOP - HEIGHT STATE LOGIC
void loop() {

  // Read current altitude from BMP280
  float pressure = bmp280.readPressure();  // in Pascals
  pressure = pressure / 100.0;  // Convert to hPa

  // Calculate altitude
  current_altitude = 44330.0 * (1.0 - pow(pressure / 1013.25, 0.1903));
  current_altitude = current_altitude - ground_altitude;  // Relative alt

  // STATE MACHINE
  switch (current_state) {

    case PRE_LAUNCH:
      // CONTINUOUSLY CHECK IF HEIGHT REACHED
      if (current_altitude >= TARGET_ALTITUDE) {
        // HEIGHT REACHED - TRANSITION TO AT_HEIGHT STATE
        current_state = AT_HEIGHT;
        height_reached_time = millis();  // Record timestamp
        Serial.println("ALTITUDE 120m REACHED!");
        sendLoRaMessage("AT_HEIGHT");

        // Log event
        logData("State: AT_HEIGHT, Time: " + String(millis()));
      }
      break;

    case AT_HEIGHT:
      // 10-SECOND COUNTDOWN TIMER
      unsigned long time_at_height = millis() - height_reached_time;

      // Send countdown status every second
      if (time_at_height % 1000 == 0) {
        int seconds_remaining = 10 - (time_at_height / 1000);
        sendLoRaMessage("T-" + String(seconds_remaining) + "s");
      }

      // CHECK IF 10 SECONDS HAVE ELAPSED
      if (time_at_height >= RELEASE_DELAY && !release_triggered) {
        // TRIGGER RELEASE MECHANISM
        releasePayload();
        release_triggered = true;

        // TRANSITION TO DESCENT_DETECT
        current_state = DESCENT_DETECT;
        logData("State: DESCENT_DETECT, Released at: " + String(millis()));
      }
      break;

    // ... other states ...
  }

  delay(100);  // 10 Hz loop rate
}

// RELEASE MECHANISM FUNCTION
void releasePayload() {
  Serial.println("RELEASING PAYLOAD NOW!");

  // Move servo to open position (90 degrees)
  releaseServo.write(SERVO_OPEN);

  // Send notification to ground station
  sendLoRaMessage("RELEASE_ACTIVATED");

  // Hold open for 1 second to ensure clean separation
  delay(1000);

  // Return servo to neutral position
  releaseServo.write(SERVO_CLOSED);

  Serial.println("Payload released. Servo returned to
neutral.");
}
```

Height Detection

1. Height Detection Logic

- Every 100ms (10 Hz), the software reads pressure from BMP280
- Pressure is converted to altitude using the barometric formula
- Altitude is compared against the target: `if (altitude >= 120.0)`
- When condition is true, state immediately transitions to `AT_HEIGHT`

2. State Transition

- PRE_LAUNCH → (altitude >= 120m) → AT_HEIGHT

3. 10-Second Countdown

- When `AT_HEIGHT` is entered, record timestamp: `height_reached_time = millis()`
- Calculate elapsed time: `time_at_height = millis()
- Height_reached_time` - Every second, send countdown to ground station ("T-10s", "T-9s", etc.)
- When `time_at_height >= 10000` (10 seconds), trigger release

Height Detection

4. Release Mechanism Activation

- Call `releasePayload()` function
- Servo moves from 0° (closed) to 90° (open)
- PWM signal on pin D3 controls SG90 servo
- Release latch opens, satellite separates from U-bolt
- Servo holds for 1 second, then returns to 0°
- Prevents accidental reactivation with `release_triggered` flag

5. Safety Features

- Once 120m is reached, won't re-trigger if altitude fluctuates
- **One-time activation:** `release_triggered` flag ensures servo only activates once
- **Ground station notification:** Every state change is transmitted via LoRa
- **Data logging:** All events logged with timestamps for post-flight analysis

6. Edge Cases Handled

- **Never reaches 120m:** If altitude never reaches target, release never happens (manual override available)
- **Altitude drops before 10s:** State remains `AT_HEIGHT`, countdown continues (assumes momentary pressure fluctuation)
- **Power loss during countdown:** On reboot, system initializes (release won't happen - fail-safe)

Sensor Data and Transmission

Summary - After Landing:

1. **Sensor Sampling:** Reduced from 10 Hz to 1 Hz (saves power)
2. **GPS:** Turned OFF (saves 45 mA)
3. **Transmissions:** - Sends **ONE final "LANDED" status packet** - Then **STOPS all transmissions**
4. **Power Management:** Enters low-power idle mode
5. **Why stop?** - Mission is complete - Conserve battery (payload may sit on ground for hours before recovery) - No useful data to transmit while stationary

Landing Confirmation Criteria:

- Altitude stable within ±0.5m for 5 consecutive seconds
- OR Accelerometer detects landing impact (sudden deceleration in Z-axis)
- Whichever occurs first triggers LANDING state

Sensor Data and Transmission

```cpp
// GLOBAL VARIABLES FOR SENSOR DATA
float altitude_m = 0.0;
float gps_latitude = 0.0;
float gps_longitude = 0.0;
float rotation_rev_per_s = 0.0;
unsigned int packet_sequence = 0;

float altitude_buffer[3] = {0, 0, 0};  // Circular buffer for 3 readings
int buffer_index = 0;


// SENSOR DATA COLLECTION (Called every 100ms = 10 Hz)
void collectSensorData() {

  // 1. READ ALTITUDE from BMP280
  float pressure = bmp280.readPressure() / 100.0;  // Convert Pa to hPa
  altitude_m = 44330.0 * (1.0 - pow(pressure / 1013.25, 0.1903));
  altitude_m = altitude_m - ground_altitude;  // Relative to ground

  // Store in circular buffer for descent detection
  altitude_buffer[buffer_index] = altitude_m;
  buffer_index = (buffer_index + 1) % 3;  // Wrap around (0, 1, 2, 0...)


  // 2. READ GPS POSITION from NEO-6M (updated at 1 Hz)
  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());  // TinyGPS++ library
  }

  if (gps.location.isValid()) {
    gps_latitude = gps.location.lat();   // Decimal degrees
    gps_longitude = gps.location.lng();  // Decimal degrees
  } else {
    // GPS not locked yet, use last known position or (0, 0)
    Serial.println("WARNING: GPS fix lost");
  }

  // 3. READ ROTATION RATE from MPU6050
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  // Get gyroscope Z-axis (yaw rotation) in degrees/second
  float gyro_z_deg_per_s = g.gyro.z;  // Already in °/s from sensor

  // Convert to revolutions per second
  rotation_rev_per_s = gyro_z_deg_per_s / 360.0;

  // Absolute value (we care about rotation speed, not direction)
  rotation_rev_per_s = abs(rotation_rev_per_s);


  // 4. LOG DATA (optional, for post-flight analysis)
  logToSD(altitude_m, gps_latitude, gps_longitude, rotation_rev_per_s);
}
```

Sensor	Sampling Rate	When
BMP280	10 Hz (every 100ms)	Continuous during all states
NEO-6M GPS	1 Hz (every 1s)	Continuous (GPS module updates at 1 Hz)
MPU6050	10 Hz (every 100ms)	Continuous during all states

Sensor Data and Transmission

```cpp
// PACKET STRUCTURE (total ~50 bytes)
struct TelemetryPacket {
  uint8_t  header[2];         // 0xAA, 0xBB (packet identifier)
  uint16 t packet seq;        // Packet sequence number (0, 1, 2, ...)
  float    altitude;          // Altitude in meters
  float    gps lat;           // GPS latitude (decimal degrees)
  float    gps_lon;           // GPS longitude (decimal degrees)
  float    rotation_rate;     // Rotation rate (rev/s)
  uint32 t timestamp;         // Milliseconds since power-on
  uint8_t  checksum;          // Simple checksum for error detection
};
```

Transmission Function:

```cpp
// TRANSMIT TELEMETRY PACKET VIA LORA
void transmitTelemetry() {

  // 1. CREATE PACKET
  TelemetryPacket packet;
  packet.header[0] = 0xAA;
  packet.header[1] = 0xBB;
  packet.packet seq = packet sequence++;  // Increment sequence number
  packet.altitude = altitude m;
  packet.gps lat = gps latitude;
  packet.gps_lon = gps_longitude;
  packet.rotation_rate = rotation_rev_per_s;
  packet.timestamp = millis();  // Milliseconds since startup

  // Calculate simple checksum (sum of all bytes % 256)
  packet.checksum = calculateChecksum(&packet);


  // 2. CONVERT TO BYTE ARRAY
  uint8 t packet bytes[sizeof(TelemetryPacket)];
  memcpy(packet_bytes, &packet, sizeof(TelemetryPacket));
  // 3. TRANSMIT VIA LORA (using LoRa library)
  LoRa.beginPacket();
  LoRa.write(packet_bytes, sizeof(TelemetryPacket));
  LoRa.endPacket();
```

State	Transmission Frequency	What is Sent
INIT	Once	Continuous during all states
PRE_LAUNCH	1 Hz (every 1s)	Status packet: current altitude, GPS fix status
AT_HEIGHT	1 Hz during countdown	Countdown messages: "T-10s", "T-9s", etc.
DESCENT_DETECT	No transmission	Radio in RX mode (listening, saving power)
TRANSMIT	3 packets in rapid succession	Full telemetry packet (Alt, GPS, Rotation, Seq#)
LANDING	Once, then stop	Final "LANDED" status packet

Telemetry Data Display

Example CSF telemetry log example excerpt:

Packet #	Altitude (m)	Latitude	Longitude	Rotation rate (rev/s)
1	92.4m	43.6532	-79.3831	0.28
2	89.7m	43.6532	-79.3831	0.31
3	86.9m	43.6533	-79.3832	0.35

- Telemetry is displayed in the ground station GUI
- Current values are shown as live numeric readouts
- Altitude vs time and rotation rate vs time are plotted in real time
- All units are in SI units: altitude in m, rotation in rev/s, GPS in decimal degrees

Telemetry Data Diagram: GUI MOCKUP